Learning Management System — Group 07

1. Group Info

- **Group Number:** 07
- **Case Study:** Group 07 — Learning Management System (LMS)
- **Course:** Software Engineering — CS13477 — Spring 2026
- **Instructor:** *Dr. Samer Elkababji*
- **Repository:** https://github.com/tabba3o/Software-engineering-project

#	Name	Student ID
1	*Mohammad Tabba'a*	*20220005*
2	*Tala Hammouri*	*20210061*
3	*Lojain Hamdan*	*20210576*
4	*Jude Mamoon*	*20210415*

2. Overview

System Purpose

The Learning Management System (LMS) is a web-based platform that supports online teaching, learning, and academic management. It enables instructors to deliver course content, create assessments, grade submissions, and distribute feedback, while students access course resources, submit assignments, and track their academic performance. Administrators manage user accounts, course offerings, and platform operations. The system is integrated with an external Authentication System, a University Student Information System (SIS), and an Email & Notification Service to support a complete academic workflow.

Tools Used

- **Visual Studio Code (VS Code)** — primary IDE for editing diagrams, Markdown, and report files.
- **PlantUML** — text-based UML modelling for all diagrams; sources kept under `/uml/`.
- **C4-PlantUML library** — used for C4 Level 1 (Context) and C4 Level 2 (Container) diagrams.
- **Pandoc** — conversion of the Markdown report into the final PDF deliverable.
- **Git & GitHub** — version control, per-member commits, tagged releases (`v1.0.0`, `v2.0.0`).

3. Diagrams

All diagram sources are stored as `.puml` files under `/uml/` and exported to `.png` for embedding in the report.

Part I — Context

- **C4 Level 1 — System Context Diagram** (`lms_c4_l1.puml`): the LMS as a single black box with its three human actors (Student, Instructor, Administrator) and three external systems (Authentication System, University SIS, Email & Notification Service).
- **C4 Level 2 — Container Diagram** (`c4_l2_container_lms.puml`): decomposes the LMS into Web Application (React SPA), Backend API Server (Python/Django), Relational Database (PostgreSQL), and File Storage Service (Object Storage).
- **Context Activity Diagram with swimlanes** (`lms_act_context_enrollment.puml`): cross-system student enrollment process, with each swimlane being one of the four C4 L1 `«system»` participants (LMS, University SIS, Authentication System, Email & Notification Service).

Part II — Interactions

- **Composite Use Case Diagrams** — one per actor: `student_usecase.puml`, `instructor_usecase.puml`, `administrator_usecase.puml`.
- **Individual Use Case Diagrams** with `<<include>>` / `<<extend>>` relationships:
  - Student: `student_submit_assignment.puml`, `student_access_course_materials.puml`, `student_view_performance_report.puml`.
  - Instructor: `instructor_upload_lecture_material.puml`, `instructor_grade_submission.puml`, `instructor_create_assessment.puml`.
  - Administrator: `admin_enroll_student.puml`, `admin_assign_instructor.puml`, `admin_generate_report.puml`.
- **Use Case Descriptions** — tabular format, one per individual use case, embedded in the report.
- **Sequence Diagrams — High-Level (stakeholder view):** `lms_seq_hl_submit_assignment.puml`, `lms_seq_hl_grade_submissions.puml`, `lms_seq_hl_enroll_students.puml`.
- **Sequence Diagrams — Detailed (developer view):** `lms_seq_dev_submit_assignment.puml`, `lms_seq_dev_grade_submissions.puml`, `lms_seq_dev_enroll_students.puml`.

Part III — Structure

- **Class Diagram** (`lms_class_diagram.puml`): ten domain classes with attributes, operations, and three relationship kinds — generalization (User → Student / Instructor / Administrator), composition (Course owns Enrollment / Assessment; Assessment owns Submission; Submission owns Grade), and aggregation (Course contains Material).

Part IV — Behavior

The LMS is a hybrid system. Its dominant character is data-driven and a discrete event-driven slice exists in the submission lifecycle. Part IV combines

both:

- **Internal-scope Activity Diagrams with swimlanes** — the data-driven view at scenario scope:
  - `act_submit_assignment.puml` — student submission scenario (UC-S1) with deadline and validation branches.
  - `act_grade_submissions.puml` — instructor grading scenario (UC-I3) with regrade-request sub-flow.
  - `act_enroll_students.puml` — administrator enrollment scenario (UC-A1) with prerequisite, capacity, and waitlist branches.
- **State Diagram** (`submission_state.puml`) — the event-driven view: lifecycle of a Submission from `Draft → Submitted → Under Review → Graded → Released` with an optional `Regrade Requested` branch.
- **State-Stimulus Table** — tabular companion to the state diagram listing every transition with current state, stimulus, guard, next state, and action.

4. Repo Structure

```
.
+-- README.md                          # Title page of the report
+-- docs/
|   +-- se_report_group_07.md          # Full Markdown report (source)
|   \-- se_report_group_07.pdf         # Final PDF report (Pandoc output)
\-- uml/
    +-- *.puml                         # PlantUML source for every diagram
    \-- *.png                          # Rendered images embedded in the report
```

- **`/README.md`** — the team's title page; embedded as the first page of the report.
- **`/docs/`** — the report in Markdown source form and the final PDF deliverable.
- **`/uml/`** — all PlantUML sources and their rendered PNG exports; the report references these PNGs via relative paths (`uml/*.png`).

5. Contributions

Member Roles

Work was distributed across the four parts of the project, with each member owning approximately one quarter of the deliverables and contributing to report editing.

Member	Role
Jude Mamoon	Part I — C4 L1 (Context) & C4 L2 (Container) modelling; context-scope swimlane activity diagram; report editing.
Tala Hammouri	Part II — Composite and individual use case diagrams; tabular use case descriptions; sequence diagrams; report editing.
Lojain Hamdan	Part II — Sequence diagrams (HL & developer); class diagram; report editing.
Mohammad Tabba'a	Part IV — Behaviour-scope activity diagrams, state diagram, and state-stimulus table; report editing.

Commits per Team Member

Member	Number of Commits
Jude Mamoon	*11*
Tala Hammouri	*11*
Lojain Hamdan	*11*
Mohammad Tabba'a	*11*

Software Engineering Project Report

Learning Management System

Group 07 — CS13477 Software Engineering — Spring 2026 Instructor: Dr. Samer Elkababji

System Description

The Learning Management System (LMS) is a web-based platform for online teaching, learning, and academic management. Three human actors interact with the system: **Students** access course materials, submit assignments, and view performance reports; **Instructors** upload lecture materials, create assessments, grade submissions, and distribute feedback; **Administrators** manage user accounts, course offerings, enrollment, and platform-wide reporting. The platform is integrated with three external systems: an **Authentication System** (single sign-on for all users), a **University Student Information System (SIS)** (canonical student records and roster sync), and an **Email & Notification Service** (deadline reminders, submission receipts, grade-release notifications).

Internally, the LMS is composed of four containers: a **Web Application** (React single-page application) that renders the user interface; a **Backend API Server** (Python/Django) that exposes a REST API and orchestrates all domain logic; a **Relational Database** (PostgreSQL) that persists users, courses, enrollments, assessments, submissions, and grades; and a **File Storage Service** (object storage) that holds the binary artifacts — lecture materials, submission files, and generated reports.

The system is **predominantly data-driven**: every primary use case (deliver materials, submit assignment, grade submission, generate report) is a transformation pipeline that moves data through validation into persistence and out again as notifications. A discrete event-driven slice exists in the **submission lifecycle**, where a submission progresses through a finite set of states (Draft → Submitted → Under Review → Graded → Released, with an optional Regrade Requested branch). The behavior modelling in Part IV reflects this hybrid nature by combining swimlane activity diagrams (data-driven flows) with a state diagram and a state-stimulus table (event-driven slice).

The report is organised in four parts following the project rubric: **Part I — Context** (C4 Level 1, C4 Level 2, and a context-scope swimlane activity diagram), **Part II — Interactions** (use case diagrams with descriptions, plus high-level and developer sequence diagrams), **Part III — Structure** (class diagram with all required relationship kinds), and **Part IV — Behavior** (internal-scope swimlane activity diagrams, a state diagram, and a state-stimulus table).

Part I — Context

Part I establishes the boundary of the system and the wider environment it operates in. The two C4 diagrams below give the static view (system-as-black-box and the internal containers); the **context activity diagram** that follows gives the dynamic view of the cross-system business process the LMS participates in. The internal scenario activity diagrams (one per primary use case) are placed in Part IV — Behavior because they describe internal system behavior rather than external business context — see the cross-references at the end of this section.

C4 Level 1 — System Context Diagram

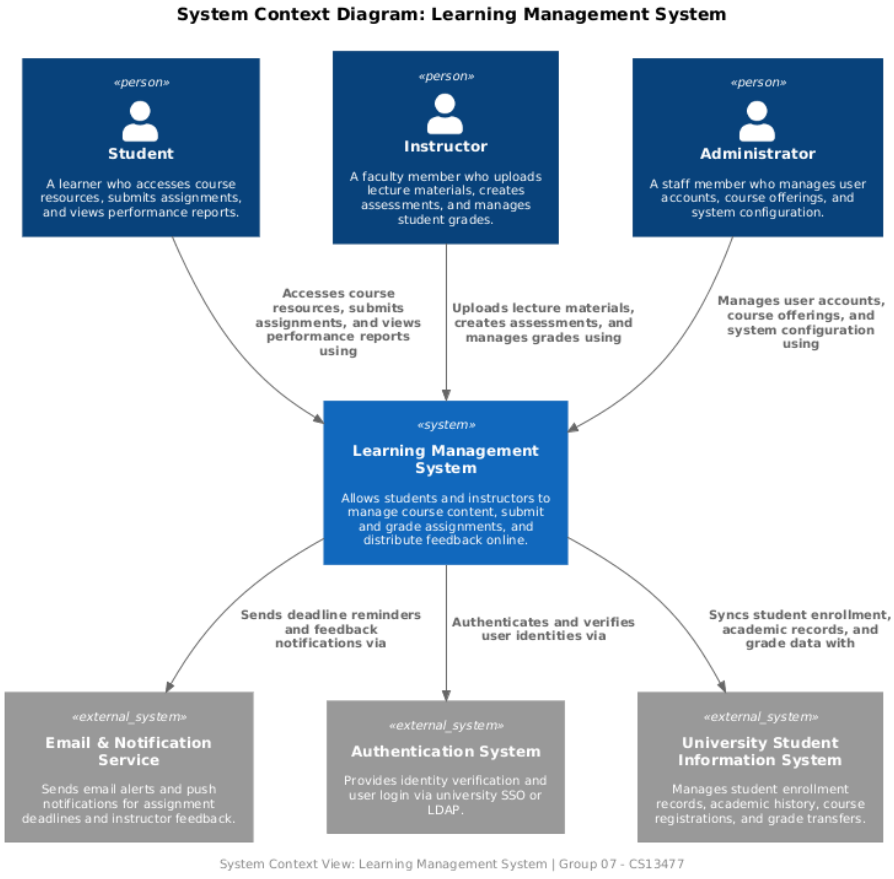

Figure 1: C4 Level 1 — System Context

The C4 Level 1 diagram shows the LMS as a single black box with the four classes of entities that interact with it. Three human actors operate the system through their browsers — **Student**, **Instructor**, and **Administrator** — each

with a different set of responsibilities. Three external systems are integrated: the **Authentication System** validates user credentials and issues session tokens; the **University SIS** is the upstream source of canonical student records and the destination for term-end grade transcripts; the **Email & Notification Service** is the outbound channel for receipts, reminders, and grade-release messages. The diagram fixes the boundary of the system and identifies every party that the LMS must integrate with — no internal containers are visible at this level. This is the basis for every subsequent diagram: the actors reappear as use case actors in Part II, the external systems reappear as the lanes of the context activity diagram below, and the boundary defined here is the boundary every other model respects.

C4 Level 2 — Container Diagram

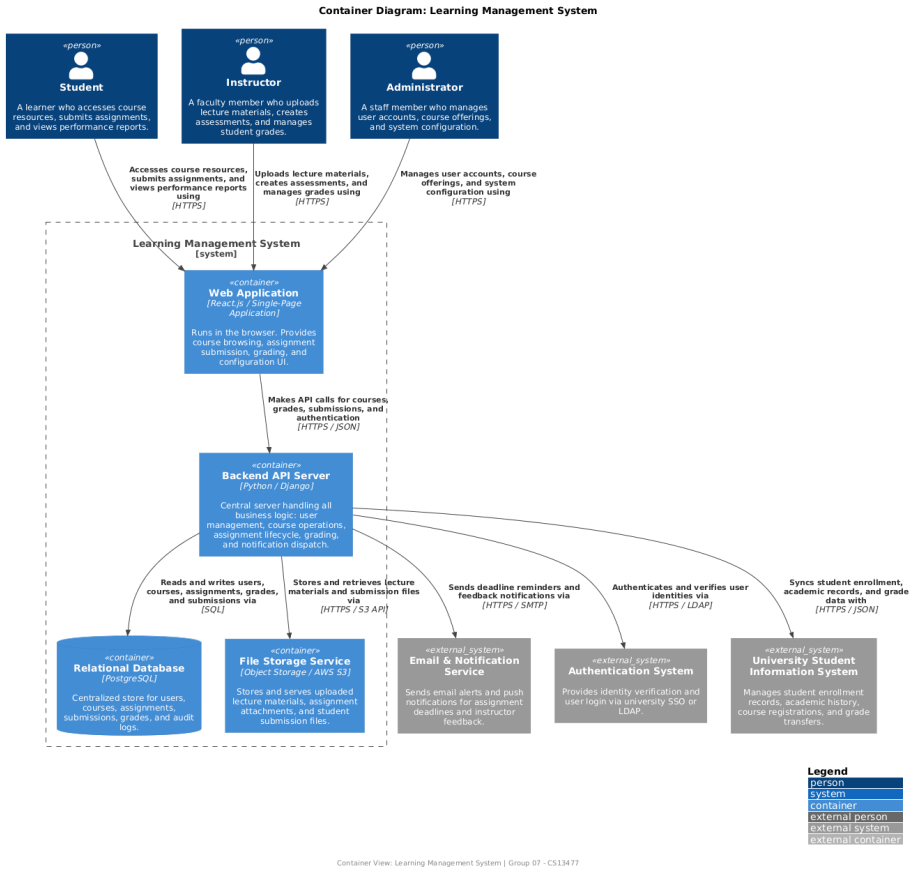

Figure 2: C4 Level 2 — Container

The C4 Level 2 diagram opens the black box from Level 1 and shows the four

internal containers that make up the LMS. The **Web Application** (React SPA, served as static assets) is the only container the user interacts with directly; it issues HTTPS calls carrying JSON to the **Backend API Server** (Python/Django). The Backend orchestrates all domain logic and talks to two persistence containers: the **Relational Database** (PostgreSQL) over a JDBC-style connection for structured data — users, courses, enrollments, assessments, submissions, grades — and the **File Storage Service** (object storage) over HTTPS for binary artifacts — lecture materials, submission files, generated reports. The three external systems from Level 1 reappear here connected to the Backend API Server: the Authentication System over an OIDC/SAML protocol, the University SIS over a REST API for roster import and grade export, and the Email & Notification Service over SMTP and a push-notification API. This is the static decomposition that the high-level and developer sequence diagrams in Part II animate.

Context Activity Diagram — Student Enrollment Process

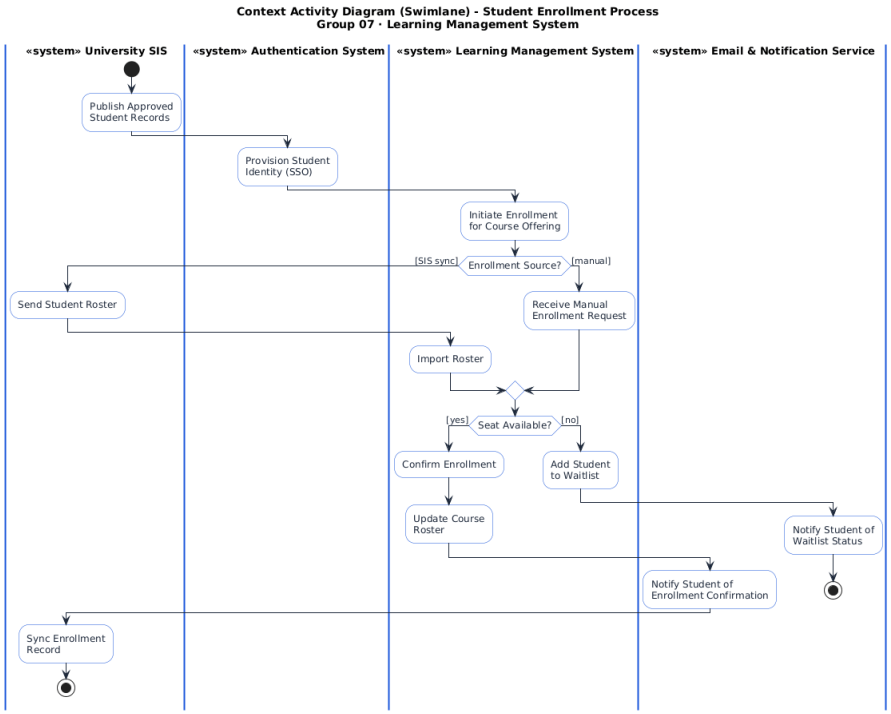

Figure 3: Context Activity Diagram — Enrollment

This is a context-scope swimlane activity diagram in the spirit of Sommerville's slide-16 Mentcare example: it shows one bounded cross-system business process and identifies the role each `«system»` participant plays. Each swimlane is one

of the four systems from the C4 Level 1 — University SIS, Authentication System, Learning Management System, Email & Notification Service. Actions are business-process verbs at organizational granularity ("Publish Approved Student Records", "Provision Student Identity", "Confirm Enrollment", "Sync Enrollment Record") rather than UI clicks or database writes. The control flow includes two business decisions — *Enrollment Source?* (SIS sync vs. manual) and *Seat Available?* (yes vs. no) — and shows the cross-lane handoffs between LMS, SIS, and Email Service that complete the process. Each terminal branch ends at its own stop node so the diagram has no auto-merge artifacts.

Cross-references to Part IV — Behavior

The rubric for Part I lists "Activity Diagram with swimlanes". Sommerville (chapter 5, *System Modelling*) draws a line between **context-scope** activity diagrams (which model business processes the system participates in alongside other «system» participants) and **internal-scope / scenario** activity diagrams (which model how the system itself behaves). The diagram above is context-scope and belongs here. The three internal-scope activity diagrams — one per primary use case — describe internal system behavior and are placed in Part IV. They are listed below for navigation:

- Activity Diagram — Submit Assignment (Scenario) — internal flow for UC-S1 (student submission with deadline and validation branches).
- Activity Diagram — Grade Submissions (Scenario) — internal flow for UC-I3 (instructor grading loop with regrade-request sub-flow).
- Activity Diagram — Enroll Students (Scenario) — internal flow for UC-A1 (administrator enrollment with prerequisite, capacity, and waitlist branches).

Part II — Interactions

Composite Use Case Diagrams

The composite use case diagrams collect every use case for one actor onto a single diagram so the actor's overall span of responsibility is visible at a glance. Each composite diagram is supplemented by detailed individual use case diagrams further down.

Student — Composite The Student actor connects to all student-facing use cases of the LMS: **Log In**, **View Enrolled Courses**, **Access Course Materials**, **Submit Assignment**, **View Performance Report**, and **Receive Notifications**. The Email & Notification Service appears as a secondary actor on Receive Notifications. This diagram defines the surface area of the system that students touch — every other student-related artifact (sequence diagrams, activity diagrams, the Student class in Part III) refines a use case shown here.

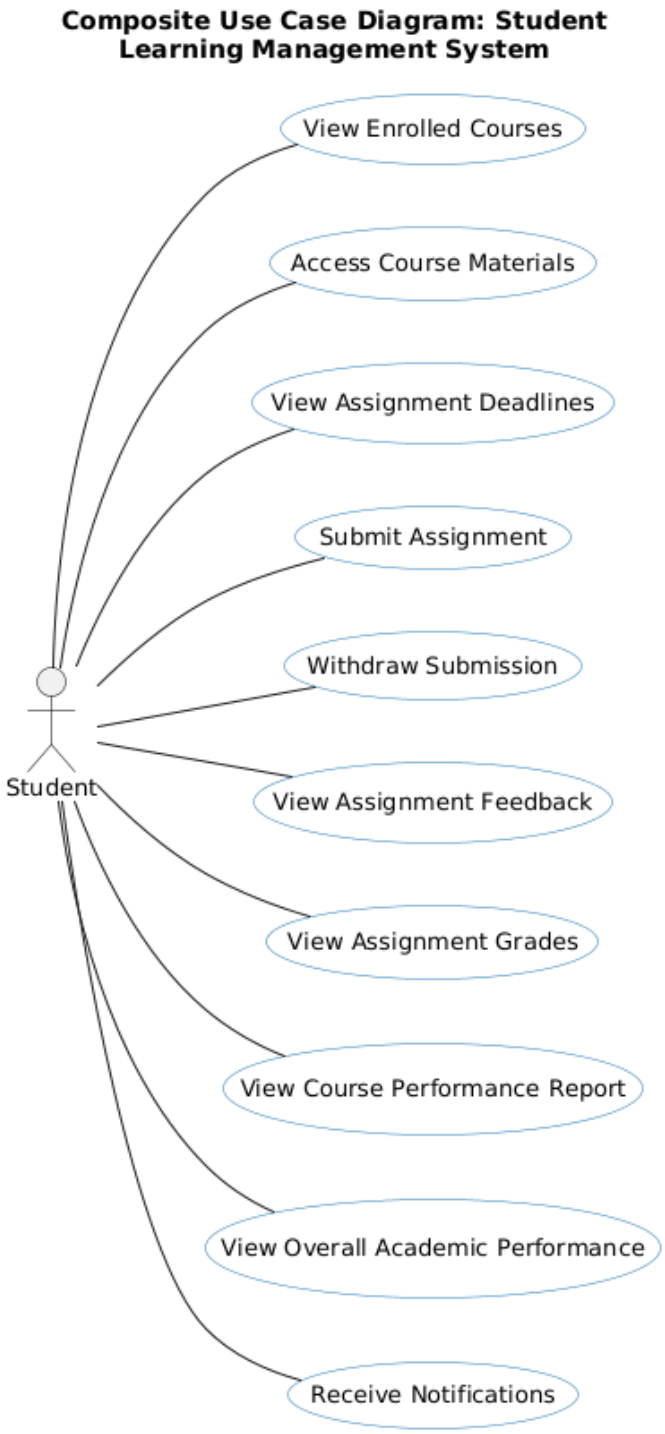

Figure 4: Student — Composite Use Cases

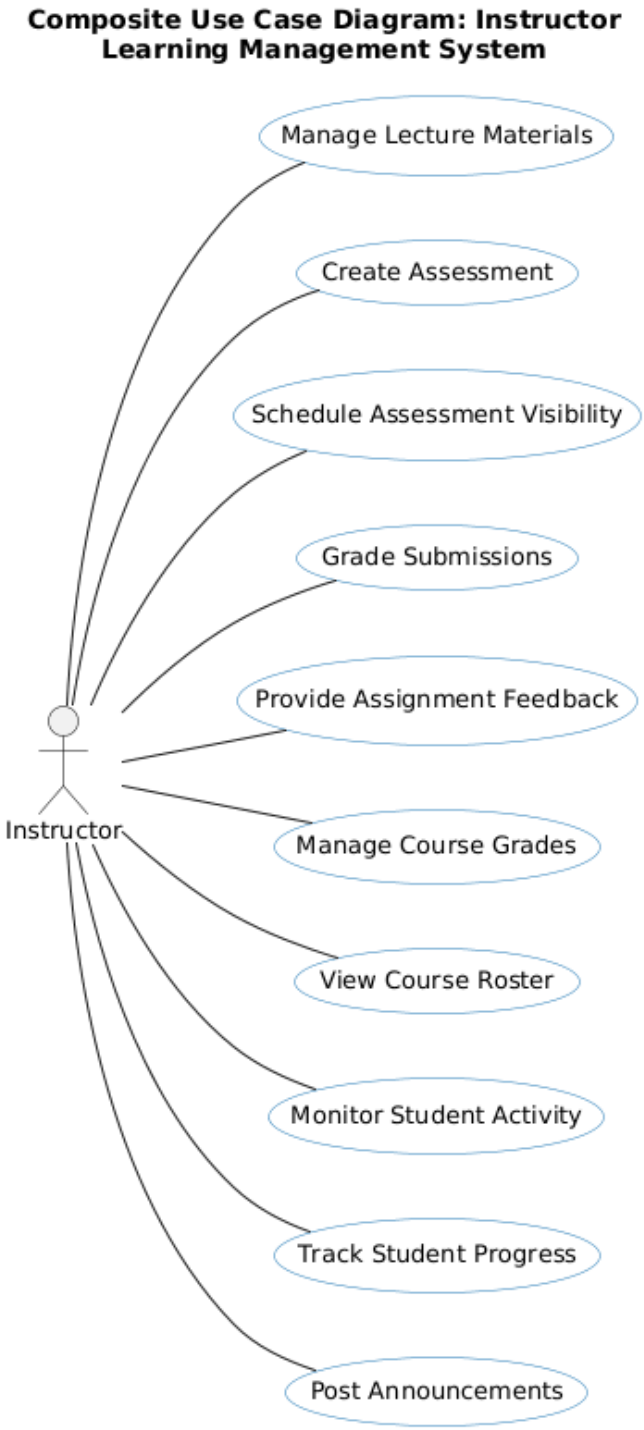

Figure 5: Instructor — Composite Use Cases

Instructor — Composite The Instructor actor connects to all teaching and assessment use cases: **Log In**, **Upload Lecture Material**, **Create Assessment**, **Grade Submission**, **Provide Feedback**, **Manage Course Roster**, and **Post Announcements**. Two of these are detailed below as individual diagrams; the rest are in scope for the system but exercised through similar flows.

Administrator — Composite The Administrator actor connects to platform-management use cases: **Log In**, **Manage User Accounts**, **Enroll Student**, **Assign Instructor to Course**, **Manage Course Offerings**, and **Generate Platform Report**. The University SIS appears as a secondary actor on Enroll Student because enrollment can be sourced either manually by the admin or via SIS roster sync.

Individual Use Case Diagrams and Descriptions

For each of the nine principal use cases below, a dedicated diagram with `<<include>>` and `<<extend>>` relationships is followed by a tabular description (Actors, Description, Data, Stimulus, Response, Comments) in the format introduced by Sommerville's *Mentcare: Transfer data* example.

UC-S1 — Submit Assignment (Student)

Field	Detail
Actors	Student (primary); Email & Notification Service (secondary)
Description	A student uploads a file as their submission against an open assessment. The system validates the file (type, size), checks the deadline, persists the submission, and dispatches a confirmation receipt and an instructor notification.
Data	Assessment ID, submission file, student ID, current timestamp
Stimulus	Student clicks *Submit* on the assessment page after attaching a file.
Response	Submission persisted with an `isLate` flag; receipt email sent to student; new-submission notification sent to instructor.
Comments	Includes *Authenticate User*. Extends *Late Submission Handling* when past deadline and late policy permits.

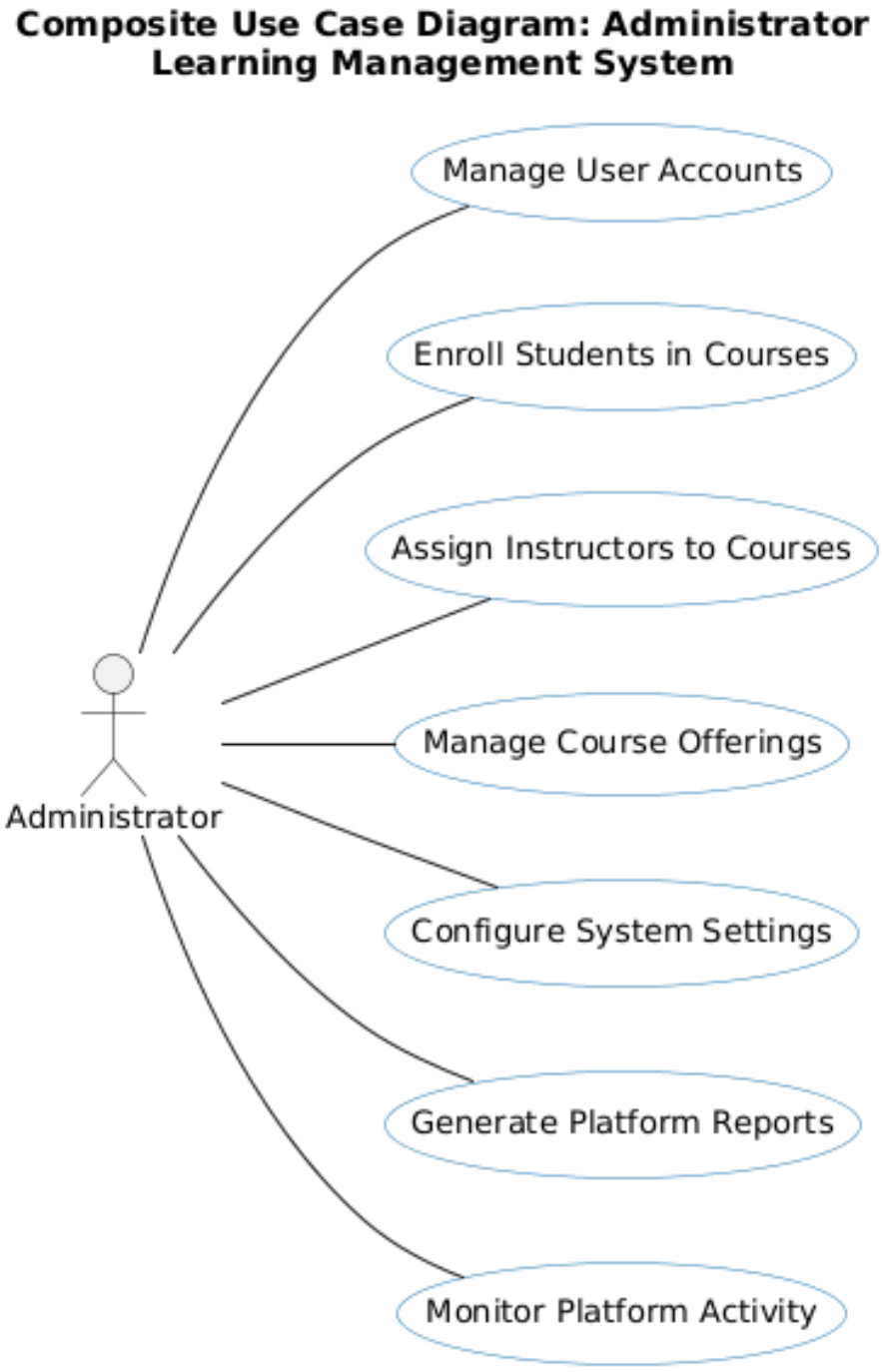

Figure 6: Administrator — Composite Use Cases

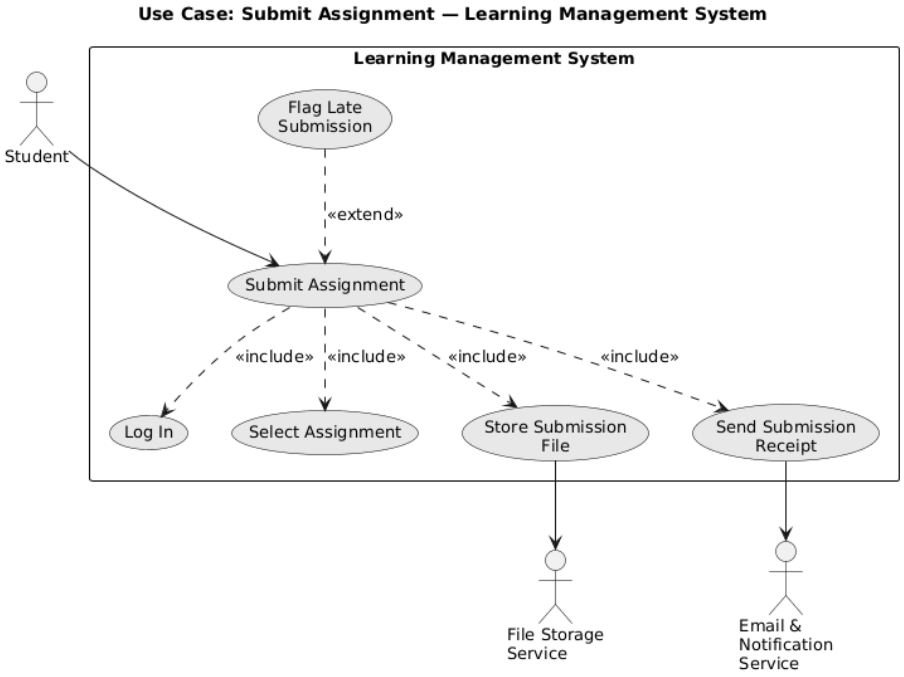

Figure 7: UC-S1 — Submit Assignment

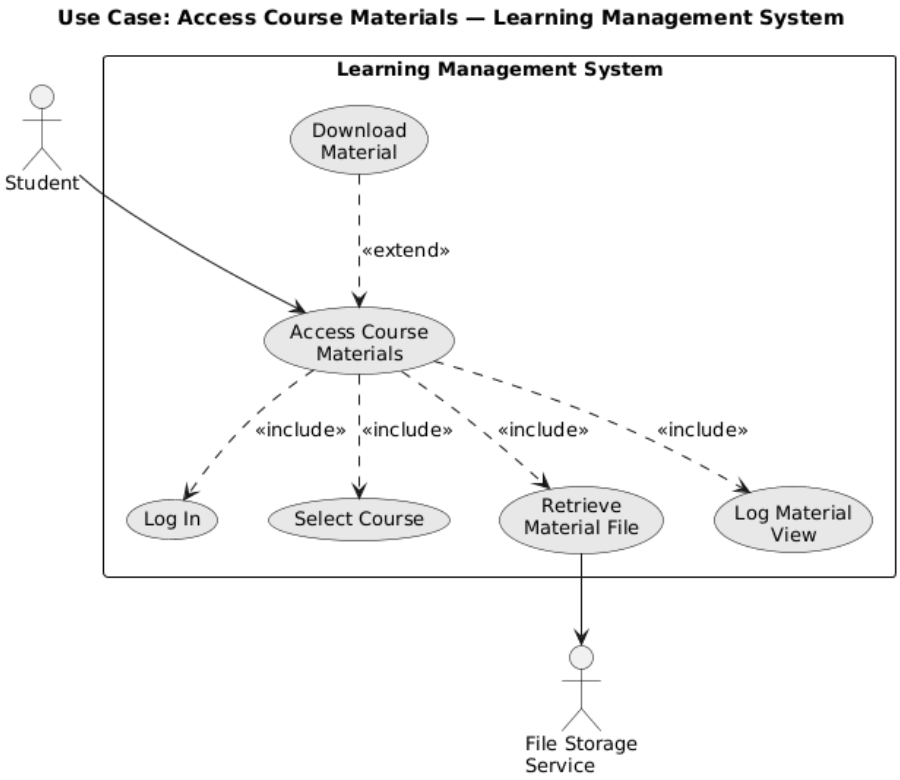

Figure 8: UC-S2 — Access Course Materials

UC-S2 — Access Course Materials (Student)

Field	Detail
Actors	Student
Description	A student browses the course page and opens or downloads a lecture material. The system serves the file from the File Storage Service after authorization.
Data	Course ID, material ID, student ID
Stimulus	Student clicks a material entry on the course page.
Response	Material file streamed from File Storage Service to the student's browser.
Comments	Includes *Authenticate User*. Extends *Track Material Access* when analytics is enabled.

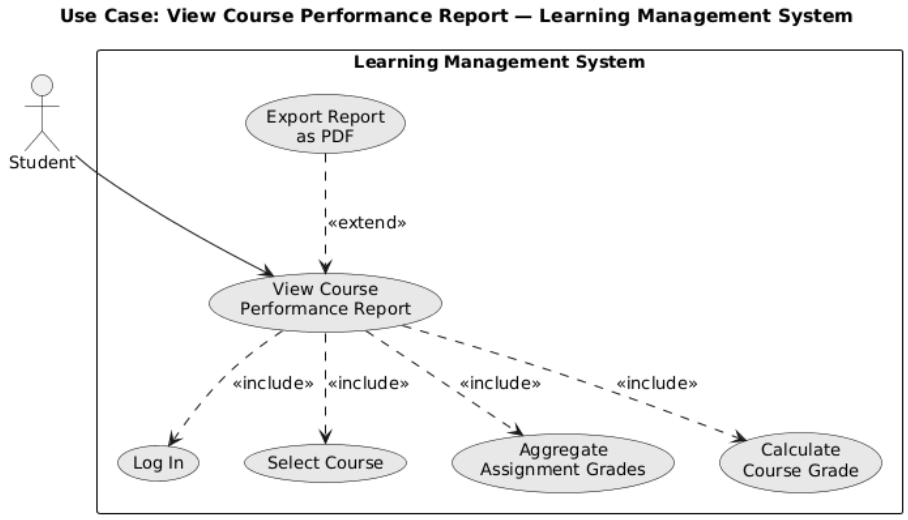

Figure 9: UC-S3 — View Performance Report

UC-S3 — View Performance Report (Student)

Field	Detail
Actors	Student

Field	Detail
Description	A student requests their performance summary across enrolled courses. The system aggregates released grades and renders the report in the browser.
Data	Student ID, term filter
Stimulus	Student opens the *Performance* tab.
Response	Report displayed in the browser showing per-course grades and overall metrics.
Comments	Includes *Authenticate User*. Only released grades are visible — submissions still under review or graded but not released are excluded.

UC-I1 — Upload Lecture Material (Instructor)

Field	Detail
Actors	Instructor
Description	An instructor attaches a lecture material (slide deck, reading, video link) to a course. The system validates the upload, stores the file in the File Storage Service, persists the metadata row, and notifies enrolled students.
Data	Course ID, material file, title, type
Stimulus	Instructor clicks *Upload Material* on the course page.
Response	Material stored and visible to enrolled students; new-material notification dispatched.
Comments	Includes *Authenticate User*. Extends *Notify Students* when the instructor opts to announce the upload.

UC-I2 — Create Assessment (Instructor)

Field	Detail
Actors	Instructor

Field	Detail
Description	An instructor authors a new assessment with title, instructions, due date, max score, and late-submission policy. The system persists the assessment and opens the submission window for enrolled students.
Data	Course ID, assessment title and instructions, due date, max score, late policy flag
Stimulus	Instructor clicks *Create Assessment*.
Response	Assessment persisted in `OPEN` state; enrolled students notified of the new assessment.
Comments	Includes *Authenticate User*.

UC-I3 — Grade Submission (Instructor)

Field	Detail
Actors	Instructor
Description	An instructor opens a submitted assignment, reviews the file, enters a grade and feedback within the assessment's max-score range, and saves. The system updates the submission status, computes per-assessment statistics, and (on release) notifies the student.
Data	Submission ID, grade value, feedback text, optional rubric attachment
Stimulus	Instructor saves a grade in the grading panel.
Response	Grade persisted; submission status advanced to *Graded* (and *Released* on subsequent release action); statistics updated; student notified on release.
Comments	Includes *Authenticate User*. The Submission state diagram in Part IV models the lifecycle this use case drives.

UC-A1 — Enroll Student (Administrator)

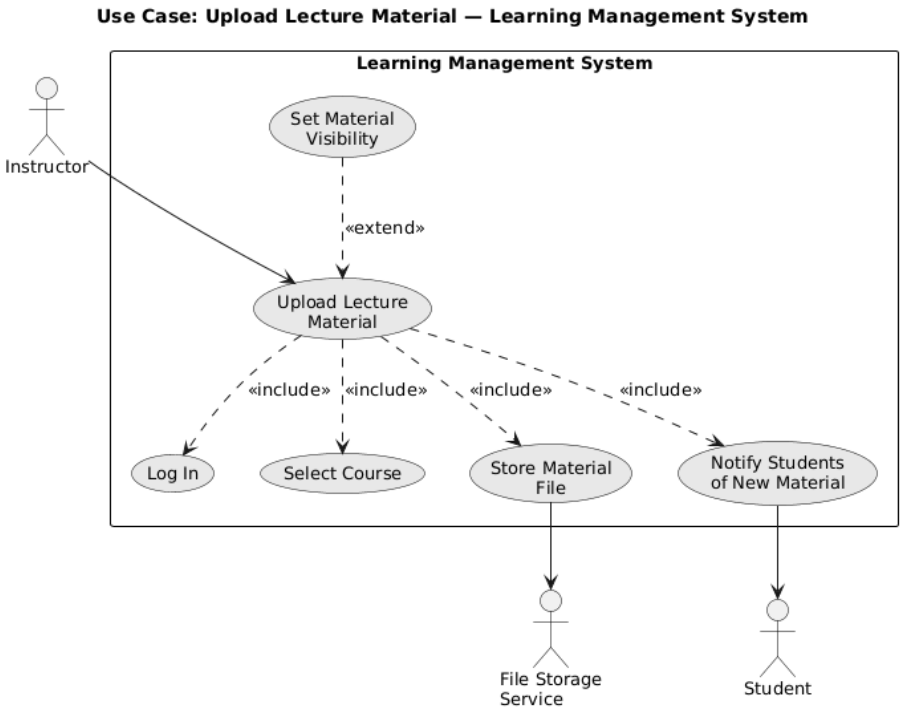

Figure 10: UC-I1 — Upload Lecture Material

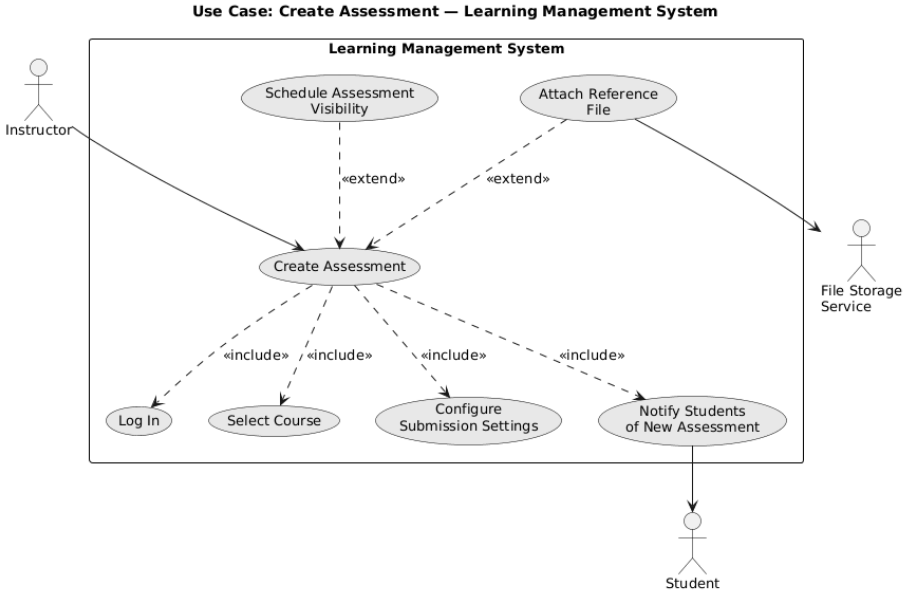

Figure 11: UC-I2 — Create Assessment

Field	Detail
Actors	Administrator (primary); University SIS (secondary)
Description	The administrator enrolls a student in a course offering — either manually by entering a student ID or in bulk via SIS roster sync. The system validates capacity and prerequisites, creates the enrollment record, and dispatches an enrollment-confirmation email.
Data	Student ID, course offering ID, source (manual / SIS), term
Stimulus	Administrator submits the enrollment form, or a scheduled SIS sync runs.
Response	Enrollment record created; student notified; SIS sync acknowledged. Waitlist entry created if capacity is full.
Comments	Includes *Authenticate Administrator*. Extends *Sync from SIS* when the source is SIS.

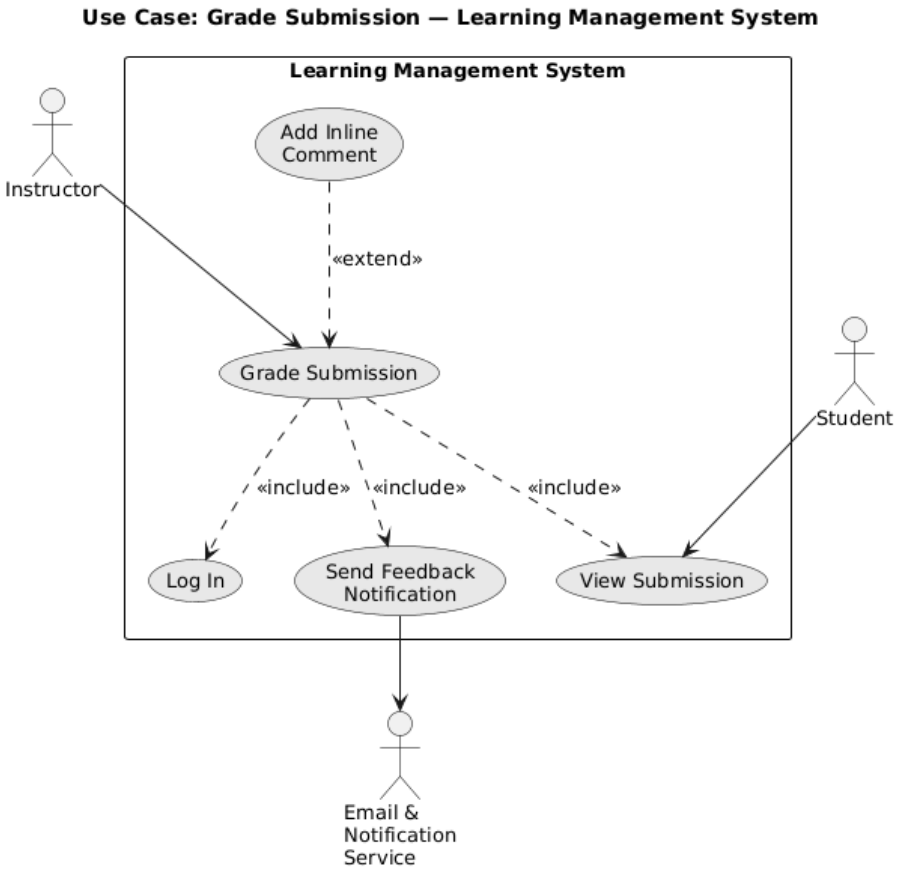

Figure 12: UC-I3 — Grade Submission

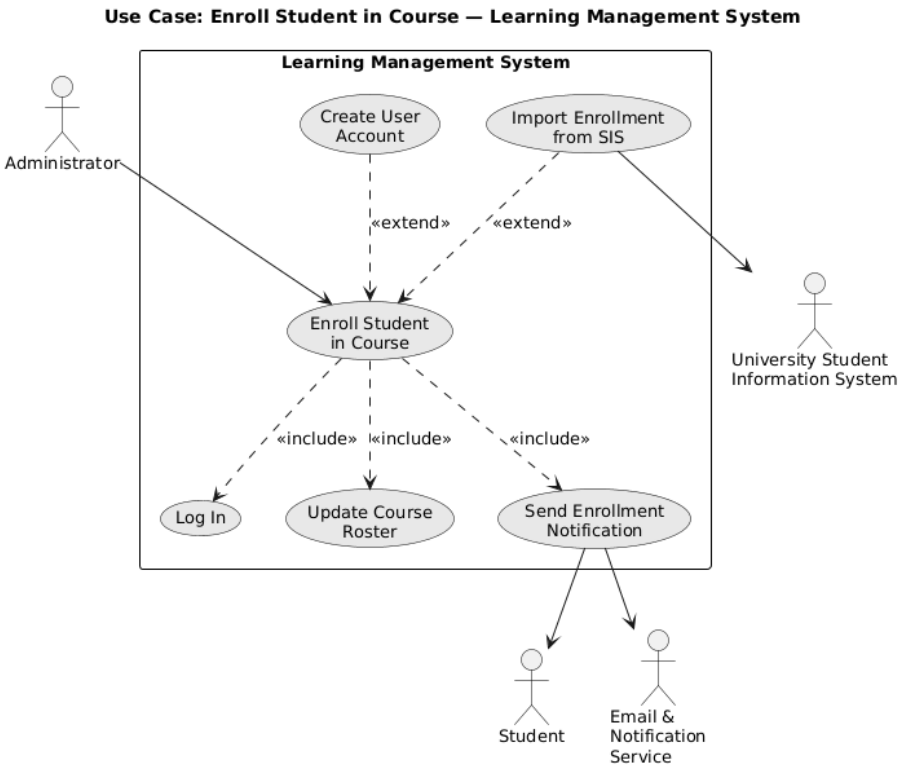

Figure 13: UC-A1 — Enroll Student

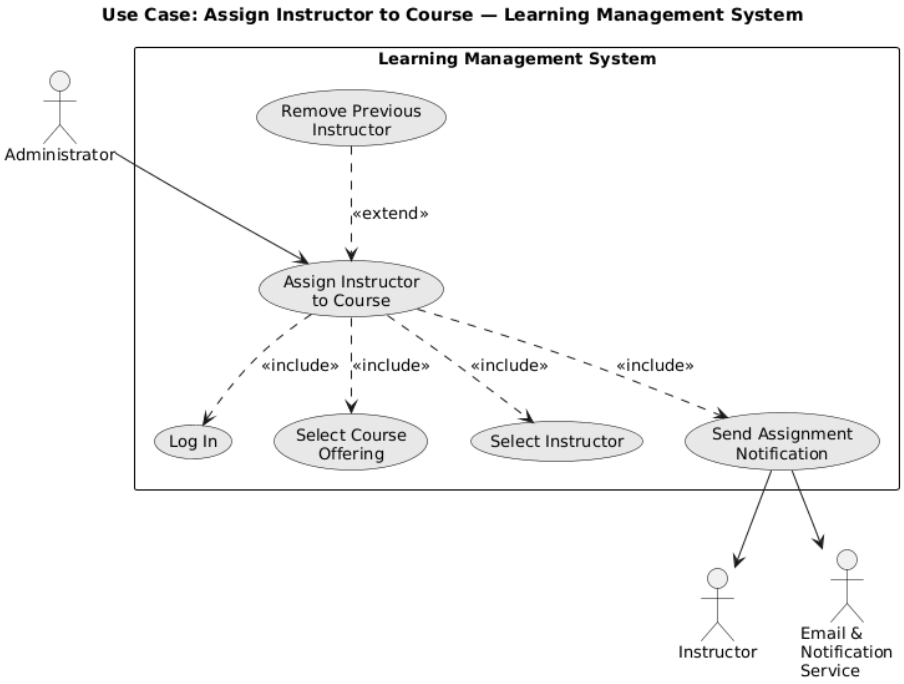

Figure 14: UC-A2 — Assign Instructor to Course

UC-A2 — Assign Instructor to Course (Administrator)

Field	Detail
Actors	Administrator
Description	The administrator assigns an instructor to a course offering for a given term. The system records the assignment and grants the instructor edit access to the course.
Data	Instructor ID, course offering ID, term
Stimulus	Administrator submits the assignment form.
Response	Assignment persisted; instructor notified by email.
Comments	Includes *Authenticate Administrator*.

UC-A3 — Generate Platform Report (Administrator)

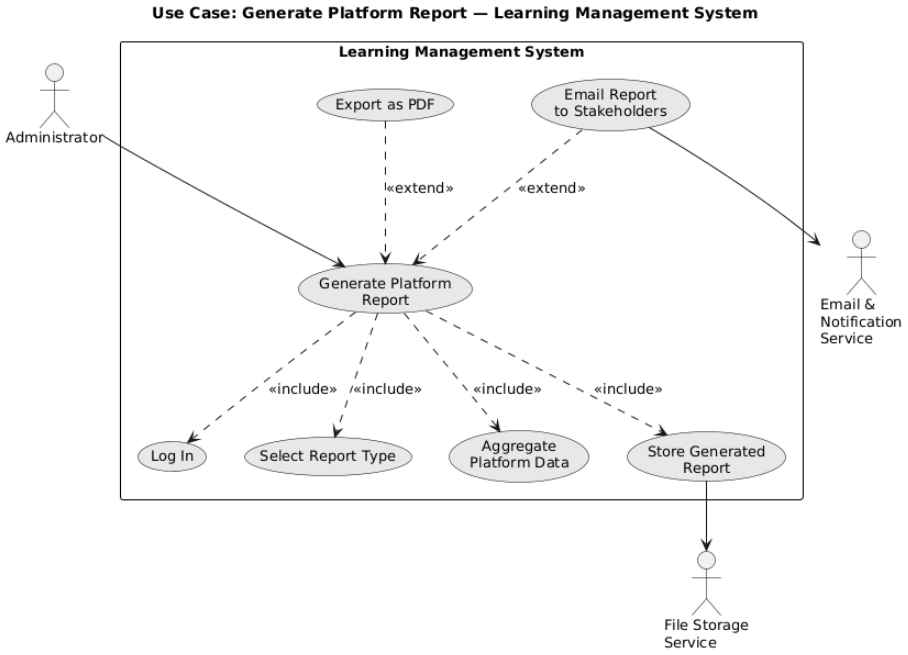

Figure 15: UC-A3 — Generate Platform Report

Field	Detail
Actors	Administrator
Description	The administrator selects a report type (usage, completion, audit) and parameters. The system aggregates data across the database, renders the report, and offers it for download or email distribution.
Data	Report type, term filter, scope filter
Stimulus	Administrator clicks *Generate*.
Response	Report rendered and stored in the File Storage Service; download link returned; optionally emailed to stakeholders.
Comments	Includes *Authenticate Administrator*. Extends *Email Report to Stakeholders* when the admin opts to distribute.

Sequence Diagrams — High-Level (Stakeholder View)

The high-level sequence diagrams show end-to-end interactions at the granularity stakeholders care about: who talks to whom, and what the user-visible outcome is. Internal containers are abstracted as a single `LMS` lifeline.

HL — Submit Assignment The student initiates the submission, the LMS validates and persists, the Email & Notification Service dispatches a receipt to the student and a new-submission alert to the instructor. The `alt` frame branches on file validity. This is the stakeholder-level view of the data path that UC-S1 describes.

HL — Grade Submissions The instructor opens the grading panel, the LMS returns the queued submissions, the instructor enters and releases a grade, and the Email & Notification Service alerts the student. The `alt` frame covers the regrade-request return path. This is the stakeholder-level view of UC-I3.

HL — Enroll Students The administrator initiates an enrollment; the LMS either pulls the roster from the University SIS (SIS-sync branch) or processes a manual entry, then writes the enrollment and dispatches a confirmation through the Email & Notification Service. This is the stakeholder-level view of UC-A1.

Sequence Diagrams — Detailed (Developer View)

The detailed sequence diagrams refine the stakeholder view by replacing the single `LMS` lifeline with the four internal containers from C4 Level 2 — Web

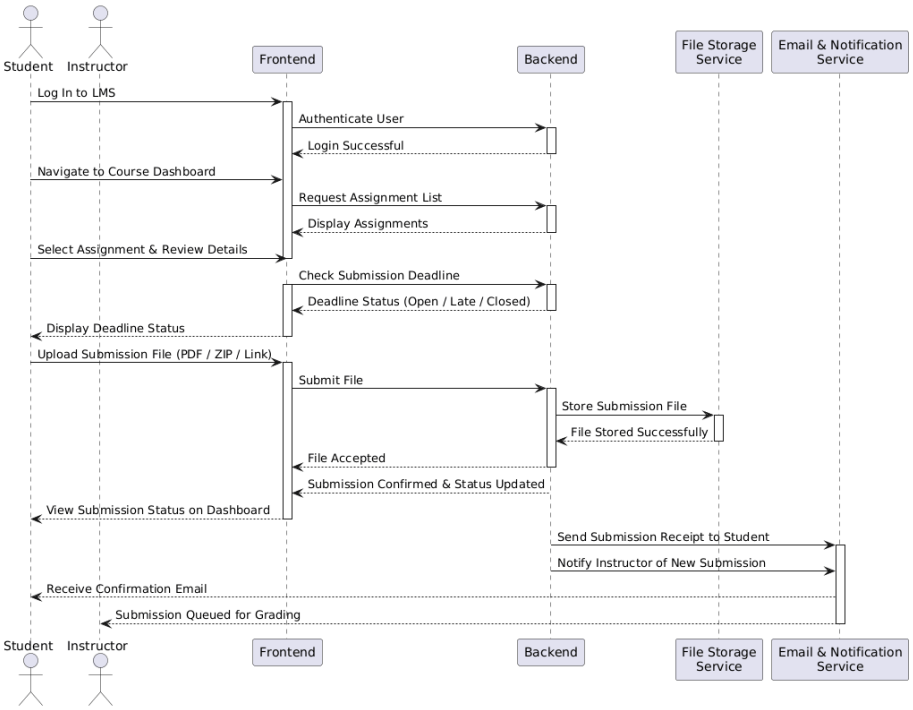

Figure 16: HL Sequence — Submit Assignment

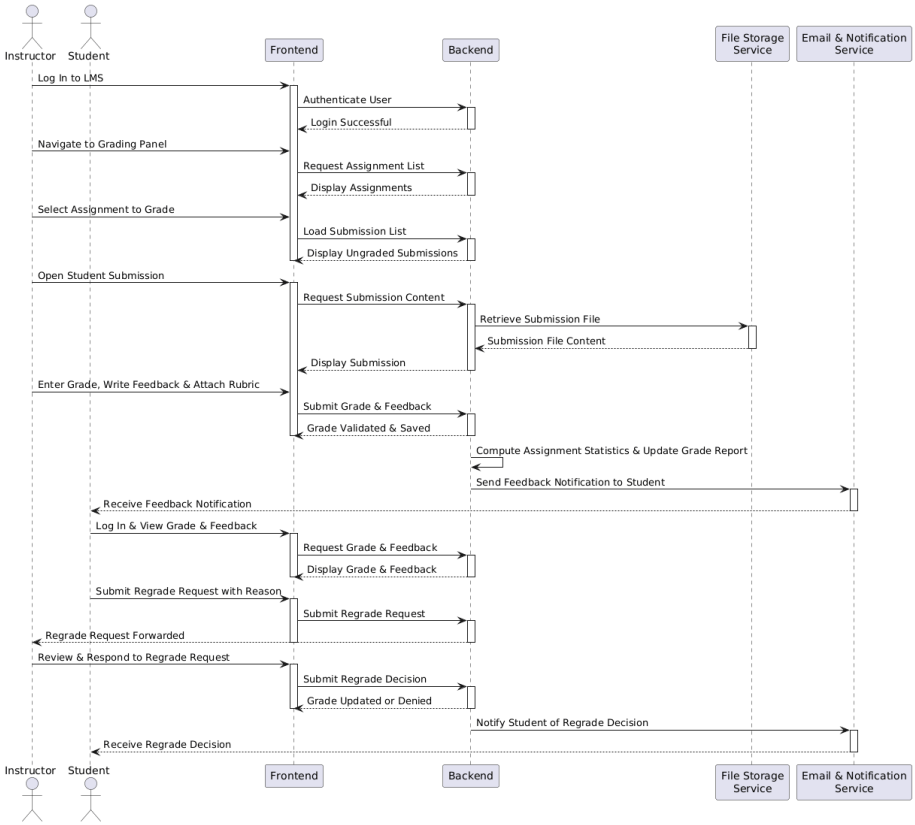

Figure 17: HL Sequence — Grade Submissions

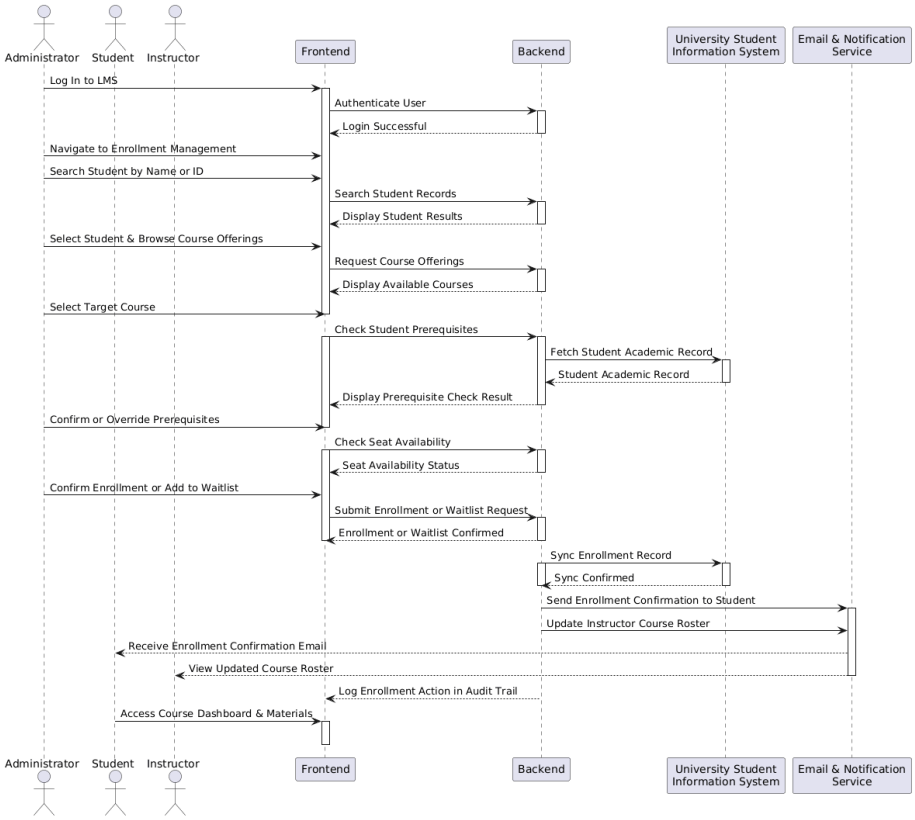

Figure 18: HL Sequence — Enroll Students

Application, Backend API Server, Relational Database, File Storage Service — and showing the per-message granularity (HTTP requests, SQL operations, S3-style PUT/GET) that developers need.

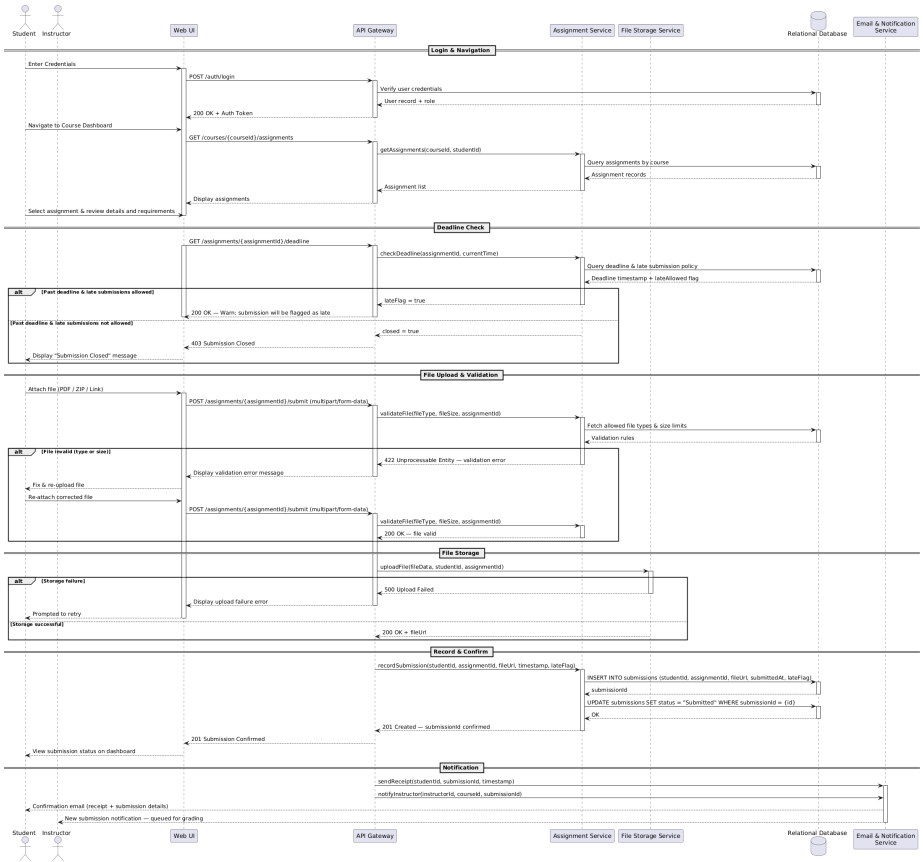

Figure 19: Dev Sequence — Submit Assignment

Dev — Submit Assignment The browser POSTs the multipart form to the Backend; the Backend validates, then in parallel writes the file to the File Storage Service (PUT) and writes the submission row to the Relational Database (INSERT); on success it triggers the Email & Notification Service. This is the developer-level realization of UC-S1 against the C4 Level 2 architecture.

Dev — Grade Submissions The grading panel issues a GET for the queued submissions (Backend SELECT), the instructor PUTs the grade payload (Backend INSERT into the `grade` table, UPDATE on `submission`), and the release action triggers the Email & Notification Service. This is the developer-level realization of UC-I3.

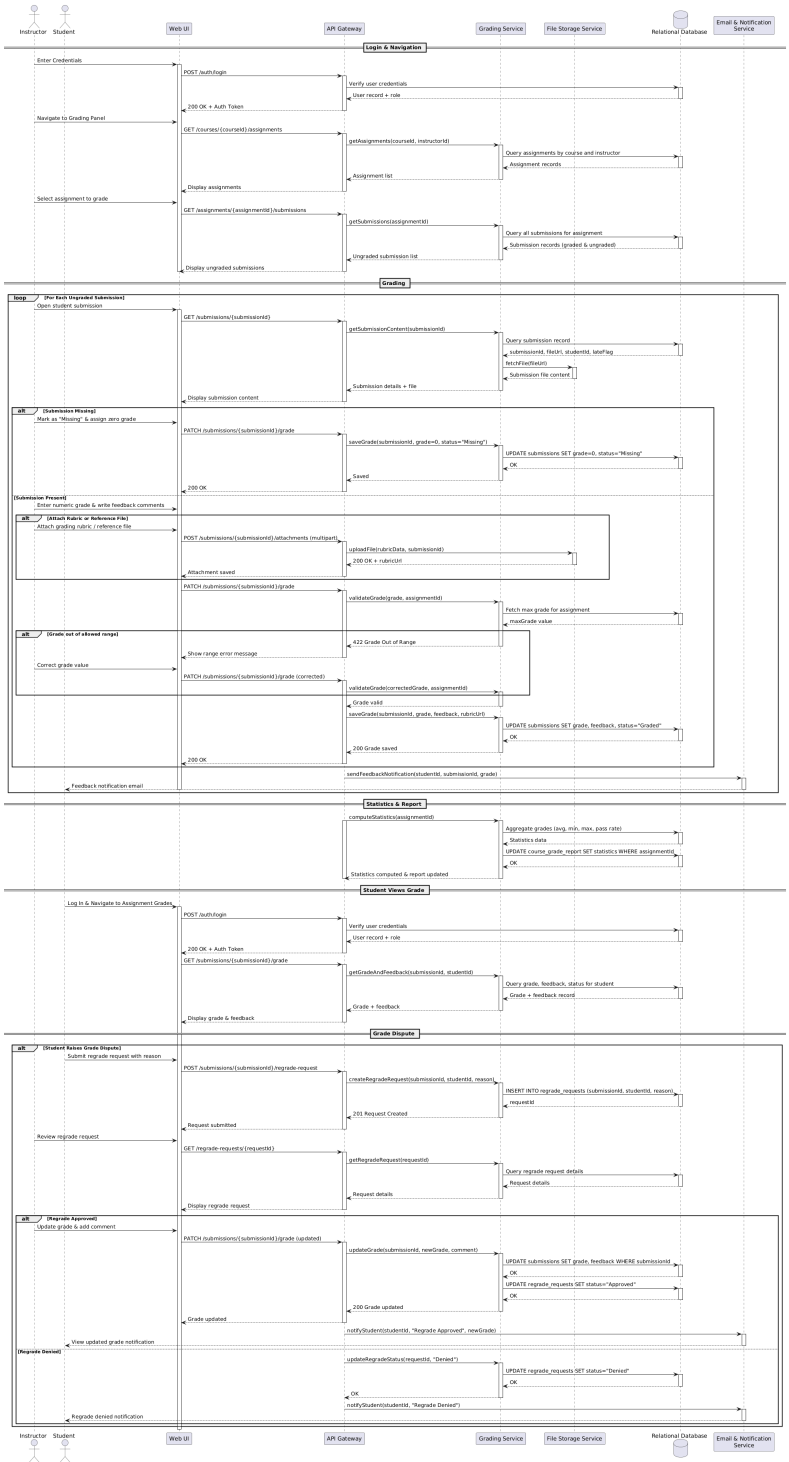

Figure 20: Dev Sequence — Grade Submissions

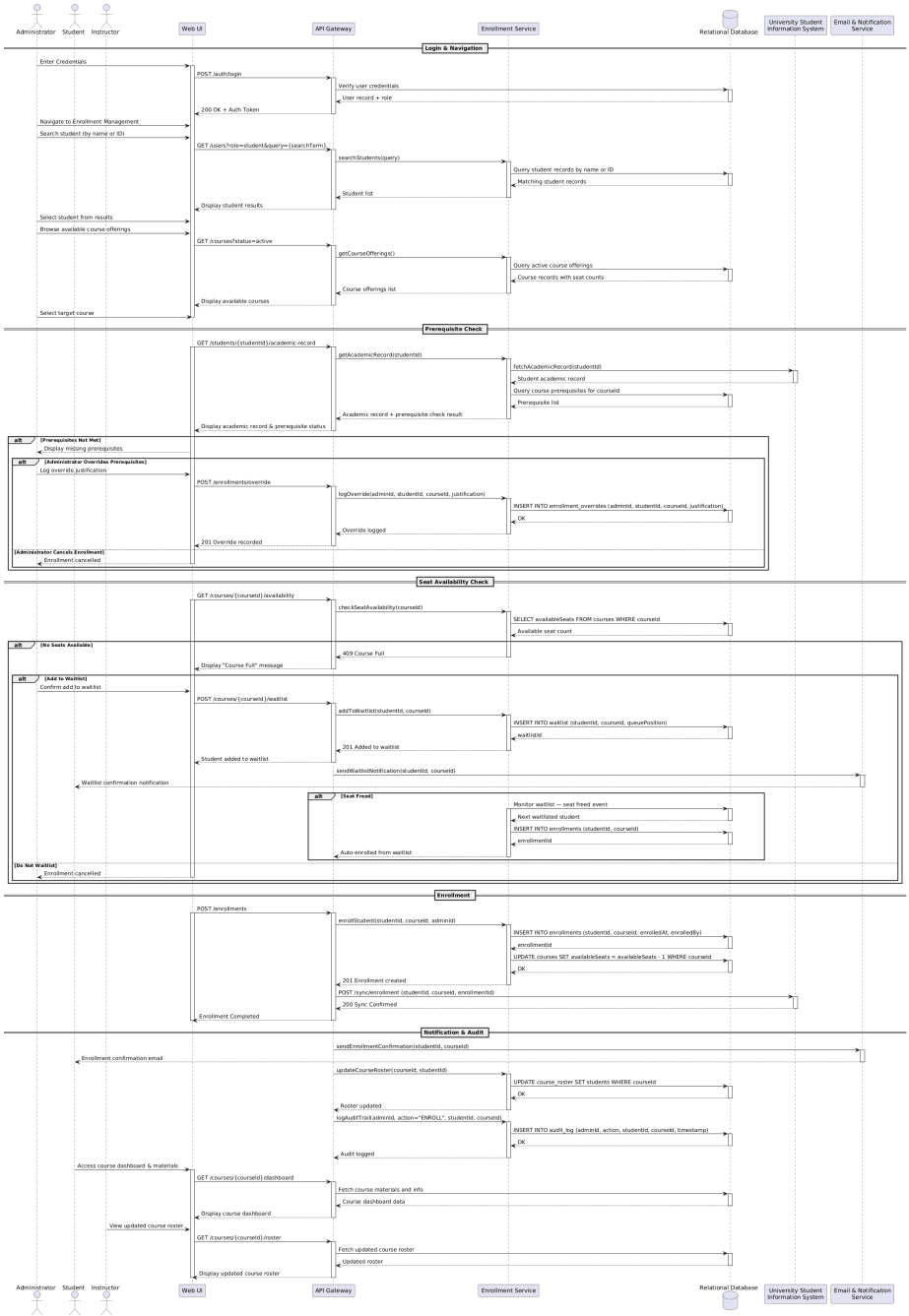

Figure 21: Dev Sequence — Enroll Students

Dev — Enroll Students The administrator UI POSTs the enrollment payload; on the SIS-sync branch the Backend issues a GET against the University SIS REST API and ingests the roster; the Backend INSERTs the enrollment rows into the database and triggers the Email & Notification Service for each student. This is the developer-level realization of UC-A1.

Part III — Structure

Class Diagram

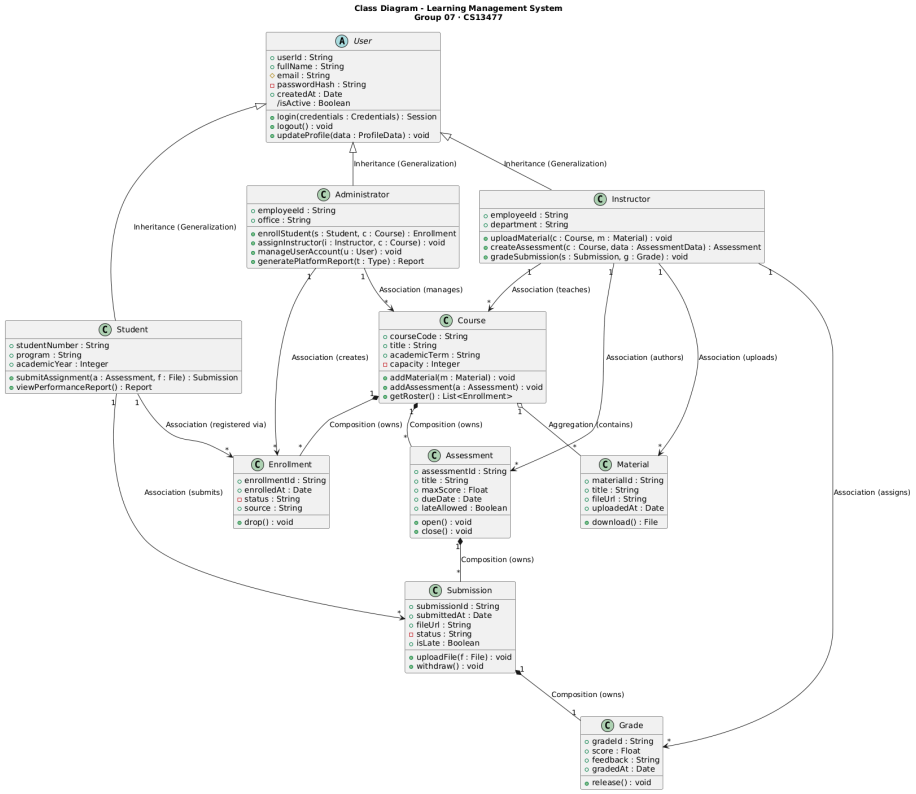

Figure 22: Class Diagram

The class diagram captures the static structure of the LMS in **ten domain classes** and demonstrates all three structural-relationship requirements from the rubric: generalization, composition, and aggregation.

Generalization (inheritance). `User` is an abstract parent class holding the common identity and authentication members (`userId`, `fullName`, `email`, `passwordHash`, `createdAt`, the derived `/isActive`, and the operations `login`, `logout`, `updateProfile`). Three specializations are de-

fined: `Student` adds `studentNumber`, `program`, `academicYear`, and the operations `submitAssignment` and `viewPerformanceReport`; `Instructor` adds `employeeId`, `department`, and the operations `uploadMaterial`, `createAssessment`, `gradeSubmission`; `Administrator` adds `office` and the operations `enrollStudent`, `assignInstructor`, `manageUserAccount`, `generatePlatformReport`. This mirrors the three actors of the C4 Level 1 and the three composite use case diagrams in Part II.

Composition (whole-part with cascade delete). Five compositions are present, all reflecting strict ownership semantics where the part has no meaning without its whole. `Course` composes `Enrollment` (an enrollment binds one student to one specific course; if the course is removed, all enrollment records go with it). `Course` composes `Assessment` (assessments are authored for one specific course, with course-bound questions and due dates). `Assessment` composes `Submission` (every submission targets exactly one assessment). `Submission` composes `Grade` (a grade exists only for the submission it evaluates). The chain `Course → Assessment → Submission → Grade` therefore cascade-deletes through the structure when a course is removed.

Aggregation (whole-part with shared / independent existence). One aggregation is present: `Course` aggregates `Material`. Lecture materials — slide decks, readings, video links — are reusable artifacts. The same slide deck can be attached to a Spring 2026 course offering and again to a Spring 2027 offering, so the material is *contained in* a course rather than *exclusively owned by* it. This is the only relationship in the diagram with the aggregation diamond rather than the filled composition diamond.

Associations. Eight plain associations capture the structural links that are neither inheritance nor whole-part. `Student → Enrollment` (registered via) and `Student → Submission` (submits) record the student's authorship of these records without owning them. `Instructor → Course` (teaches), `Instructor → Material` (uploads), `Instructor → Assessment` (authors), and `Instructor → Grade` (assigns) record the instructor's authorship and assignment role. `Administrator → Enrollment` (creates) and `Administrator → Course` (manages) record the admin's platform-management responsibilities — note that `Administrator → User` is intentionally absent because Administrator inherits from User and managing user accounts is captured as the operation `manageUserAccount`, not as a structural relationship from a subclass back to its parent.

Multiplicities follow the rubric reference style: `"1"` and `"*"` quoted on either end, with the role label and relationship kind shown on the connector.

Part IV — Behavior

The LMS is a hybrid system: predominantly data-driven (almost every primary use case is a transformation pipeline that moves data through validation into persistence) with a discrete event-driven slice in the submission lifecycle. Per

the rubric's "choose most representative model" guidance for hybrid systems, this section combines both:

- The **data-driven view** is captured by three internal-scope swimlane activity diagrams covering the three primary scenarios (§1–§3 below). The **context-scope** activity diagram (cross-system enrollment process) is placed in Part I — Context because it describes a wider business process the LMS participates in alongside its external `«system»` collaborators rather than internal system behavior.
- The **event-driven view** is captured by a state diagram and a state-stimulus table for the `Submission` entity (§4–§5 below).

Activity Diagrams (Data-Driven View — Internal Scope)

1. Activity Diagram — Submit Assignment (Scenario) This is a scenario-scope swimlane activity diagram for UC-S1 (Submit Assignment). The lanes are role/component lanes — Student, System, File Storage Service — and the actions are at UI/persistence granularity: *Log In*, *Open Assessment*, *Validate File*, *Store Submission*, *Send Receipt*. Decision branches cover deadline-passed handling (with the late-policy sub-decision) and file-validity handling (with the fix-and-re-upload loop). A fork at the end dispatches the receipt to the student and the new-submission notification to the instructor in parallel. This diagram is the data-driven complement to the high-level and developer sequence diagrams of UC-S1 in Part II.

2. Activity Diagram — Grade Submissions (Scenario) Scenario-scope activity diagram for UC-I3 (Grade Submission). Lanes are Instructor and System. The flow covers opening the grading panel, retrieving the queued submissions, the per-submission grade-and-feedback loop with grade-validity check, persisting grade and updating submission status, releasing the grade, and the regrade-request sub-flow that loops back through the instructor's review. The repeat construct over the queue and the regrade sub-flow are the two structural features that the high-level sequence diagram abstracts away.

3. Activity Diagram — Enroll Students (Scenario) Scenario-scope activity diagram for UC-A1 (Enroll Student). Lanes are Administrator, System, and University SIS. The flow covers the source decision (manual vs. SIS sync), prerequisite check, capacity check with waitlist branch, enrollment-record creation, and confirmation dispatch. This is the data-driven view that complements the use case description and sequence diagrams of UC-A1.

Event-Driven View

4. State Diagram — Submission Lifecycle The state diagram models the lifecycle of a single `Submission` entity — the part of the LMS that is genuinely

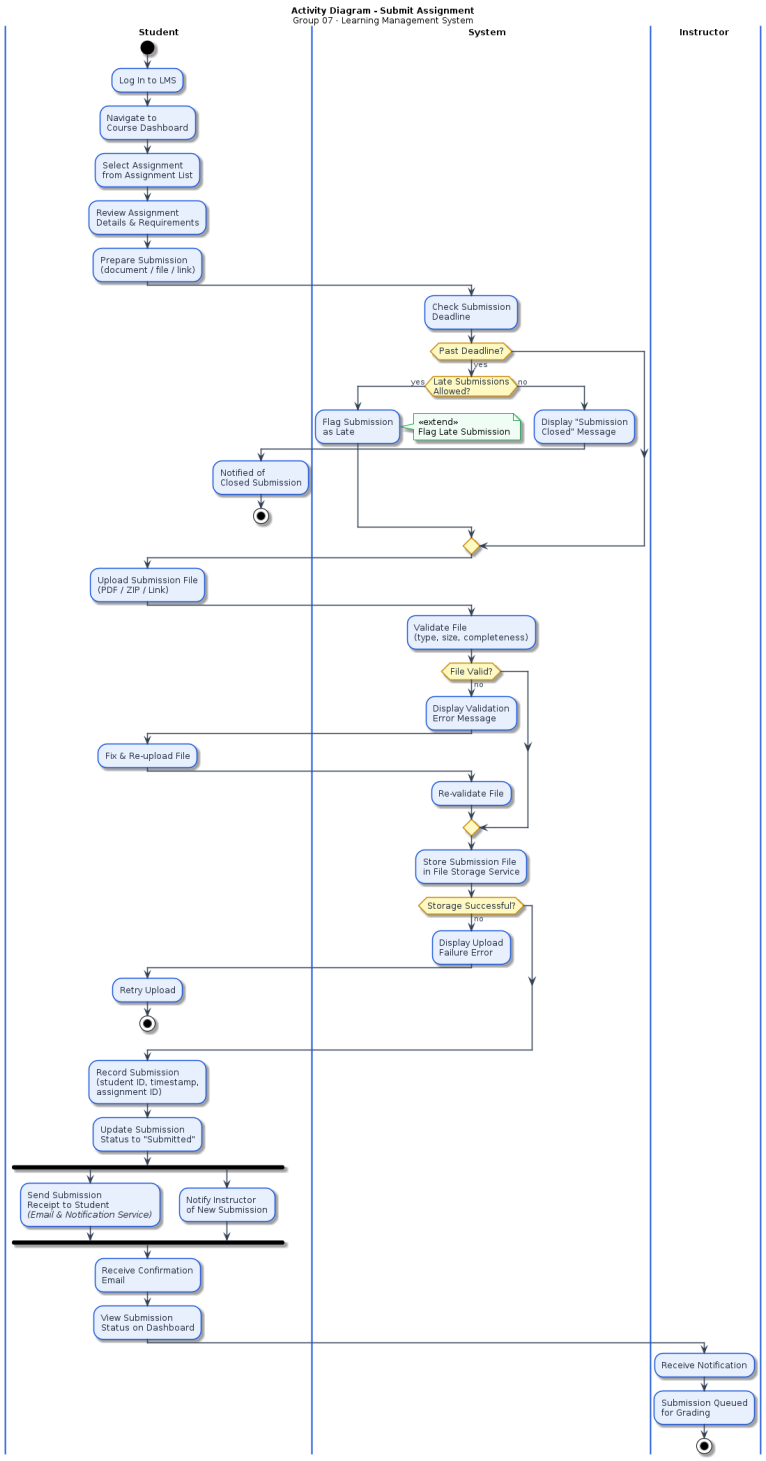

Figure 23: Activity Diagram — Submit Assignment

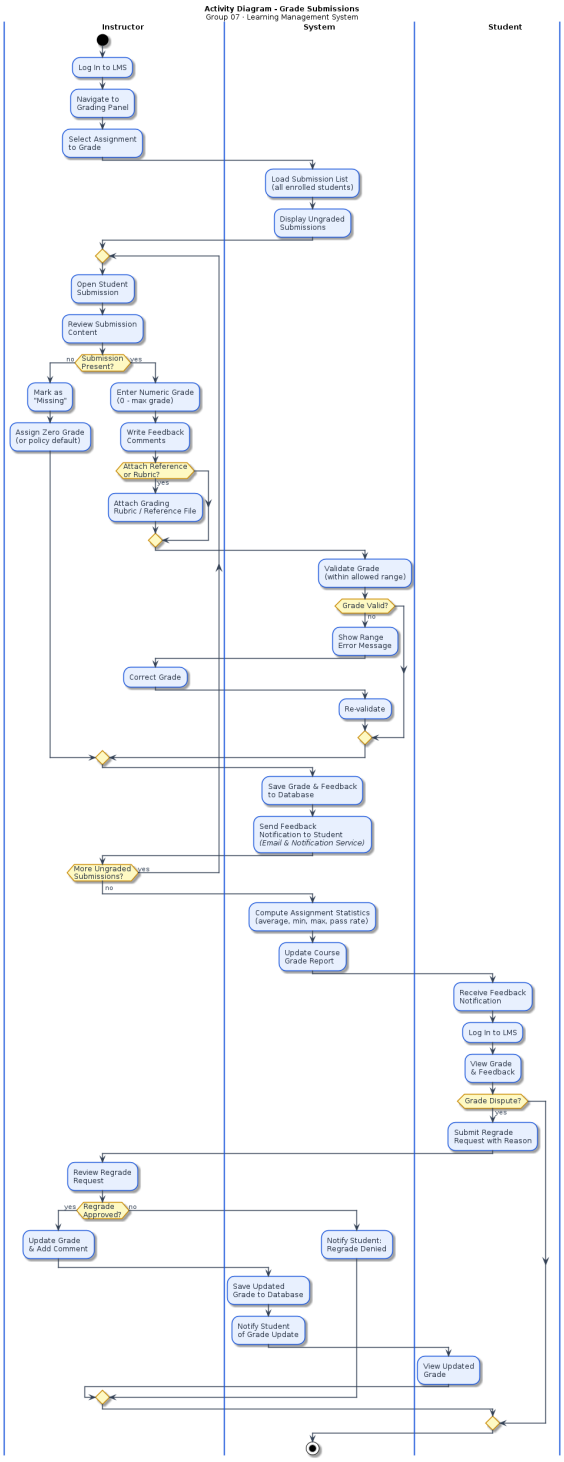

Figure 24: Activity Diagram — Grade Submissions

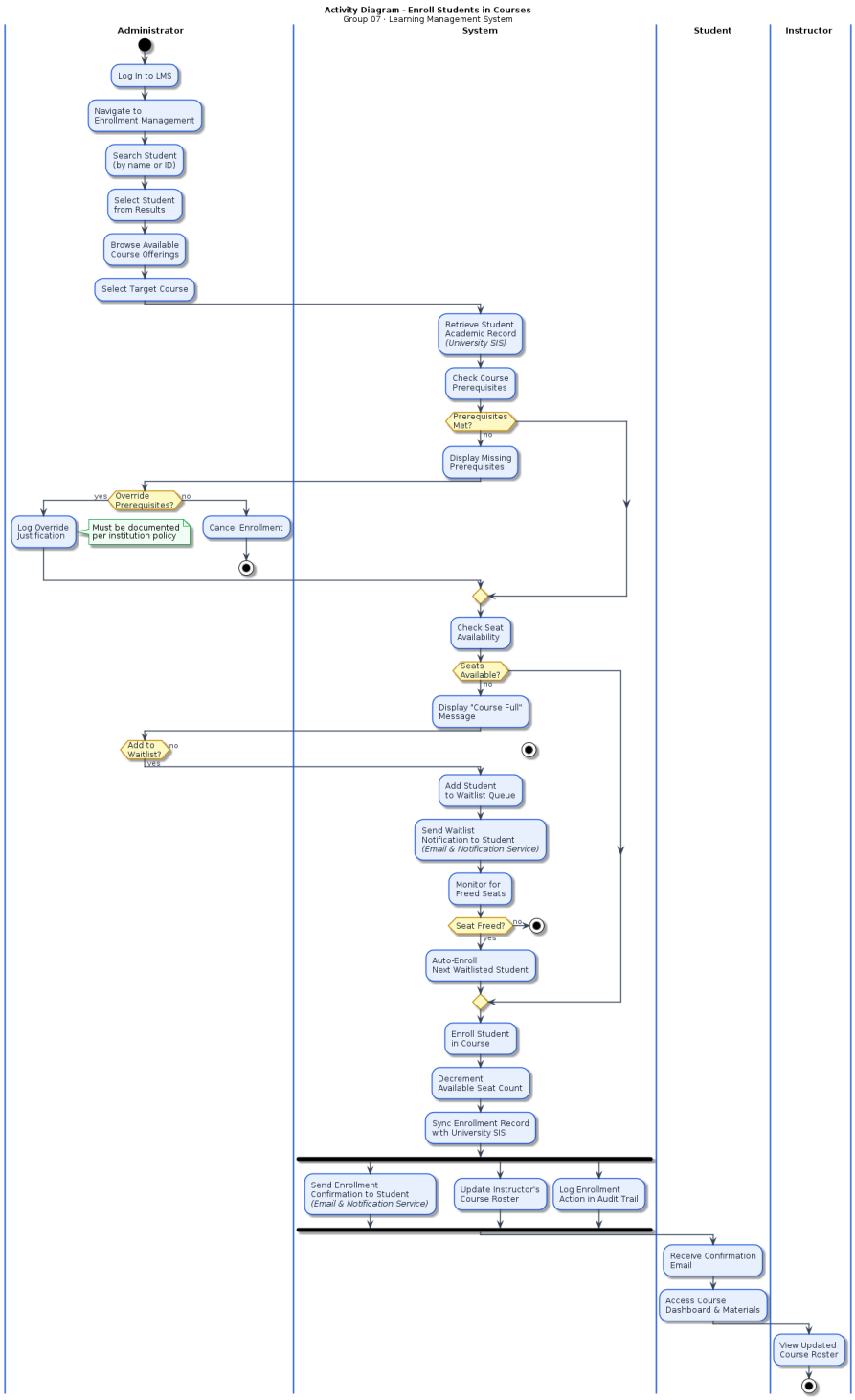

Figure 25: Activity Diagram — Enroll Students

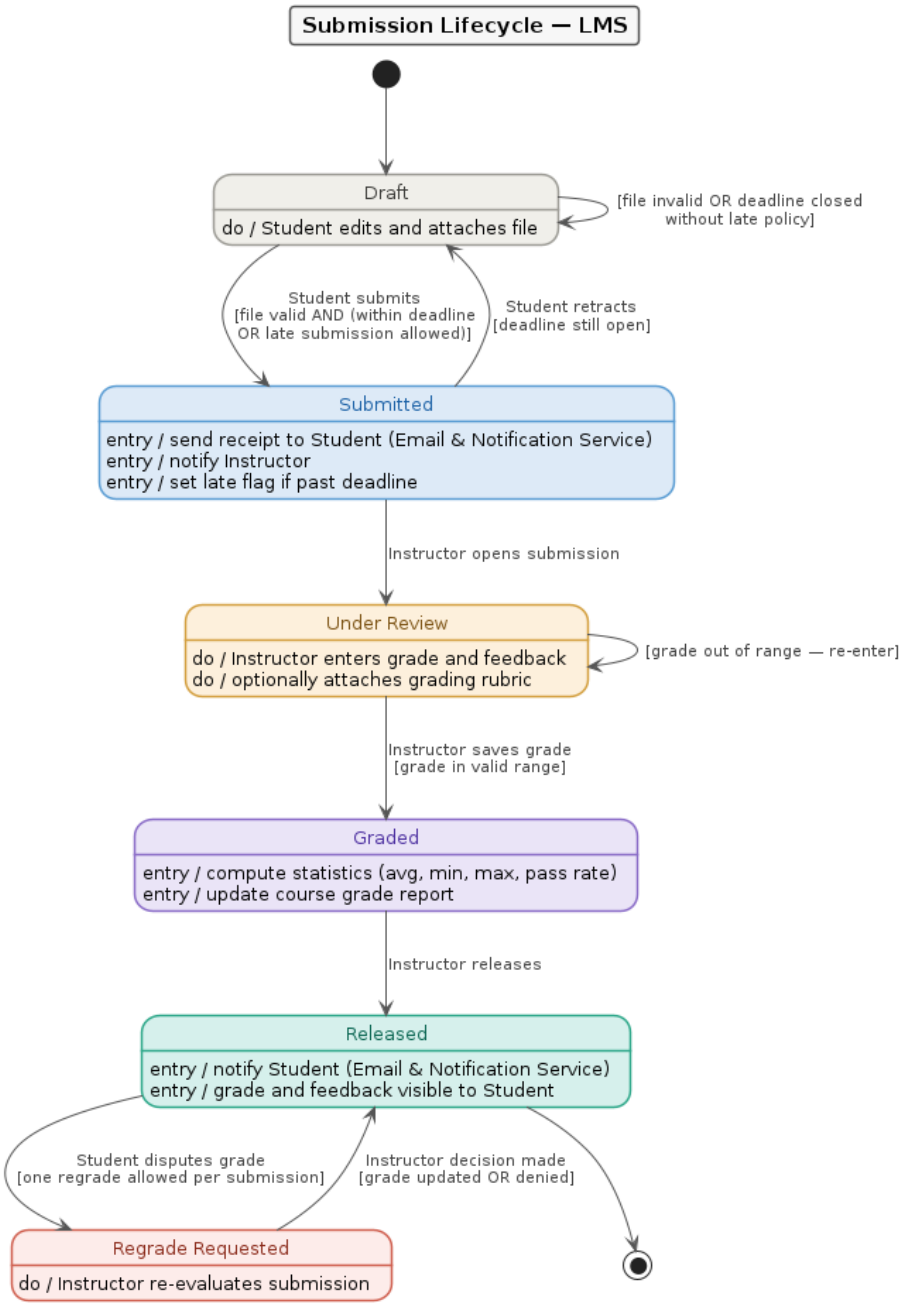

Figure 26: State Diagram — Submission Lifecycle

event-driven, in the sense that the same entity exists in a finite set of distinguishable states and transitions only on specific stimuli with guard conditions. Six states are present:

- **Draft** — the default state before any submission attempt; the student can edit and attach a file.
- **Submitted** — entered when a valid file is uploaded within the deadline (or with late policy permitting); entry actions dispatch the receipt, notify the instructor, and set the late flag.
- **Under Review** — entered when the instructor opens the submission in the grading view; the instructor enters grade and feedback and may attach a rubric.
- **Graded** — entered when a valid grade is saved; entry actions compute per-assessment statistics and update the course grade summary. The grade is not yet visible to the student.
- **Released** — entered when the instructor explicitly releases the grade; entry actions notify the student and make the grade and feedback visible.
- **Regrade Requested** — entered when the student disputes a released grade; the instructor re-evaluates and either updates the grade or denies the request, returning to Released.

Self-loops are present on Draft (file invalid, deadline closed without late policy, attempt limit exceeded) and Under Review (grade out of valid range — re-enter). One backward transition exists from Submitted to Draft when the student retracts the submission while the deadline is still open. The terminal transition is from Released to the final state when the term ends and grades are archived.

5. State-Stimulus Table The table below is the tabular companion to the state diagram in §4. It enumerates every transition with the precision the diagram cannot show: the exact stimulus (event), guard condition, next state, and action taken on the transition.

Current State	Stimulus (Event)	Guard Condition	Next State	Action / Response
Draft	Student clicks Submit	File valid AND (within deadline OR late submission allowed)	**Submitted**	Store file in File Storage Service; record submission with timestamp; send confirmation receipt to Student via Email & Notification Service; notify Instructor; set late-submission flag if past deadline
Draft	Student clicks Submit	File violates size / type constraints	**Draft**	Reject upload; display validation error; return control to Student to fix and re-upload
Draft	Student clicks Submit	Submission window closed AND late submission NOT allowed	**Draft**	Display "Submission Closed" message; reject upload
Draft	Student clicks Submit	Attempt limit exceeded	**Draft**	Reject upload; notify Student that attempt limit has been reached

Current State	Stimulus (Event)	Guard Condition	Next State	Action / Response
Submitted	Student withdraws submission	Deadline still open	**Draft**	Withdraw submission record
Submitted	Instructor selects submission in grading view	—	**Under Review**	Open submission via submission viewer
Under Review	Instructor saves grade	Grade value out of valid range	**Under Review**	Display out-of-range error; prompt Instructor to re-enter grade
Under Review	Instructor saves grade	Grade value in valid range	**Graded**	Persist grade and written feedback to database; optionally attach inline comments / grading rubric / reference file
Graded	Instructor clicks Release	—	**Released**	Compute per-assignment statistics (average, min, max, pass rate); update course grade summary; send grade-released notification to Student via Email & Notification Service

Current State	Stimulus (Event)	Guard Condition	Next State	Action / Response
Released	Student files regrade request	—	**Regrade Requested**	Record regrade request; notify Instructor
Regrade Requested	Instructor updates grade	Grade changed after re-evaluation	**Released**	Persist revised grade; send updated-grade notification to Student via Email & Notification Service
Regrade Requested	Instructor denies regrade	Original grade kept	**Released**	Send regrade-denied notification to Student via Email & Notification Service

Closing Remarks

This report covers all four perspectives required by the rubric. **Part I — Context** establishes the boundary and internal decomposition through the C4 Level 1 and Level 2 diagrams together with one context-scope swimlane activity diagram (cross-system enrollment process); the latter satisfies the rubric's "Activity Diagram with swimlanes" line and complements the static C4 view with a dynamic, business-process view of how the LMS fits among its `«system»` collaborators. **Part II — Interactions** captures the interaction perspective via use case diagrams (composite and individual), tabular use case descriptions, and sequence diagrams at two levels of detail (high-level for stakeholders, developer-level against the C4 Level 2 architecture). **Part III — Structure** captures the static structure in a single class diagram with all three required relationship kinds — generalization, composition, and aggregation — across ten domain classes. **Part IV — Behavior** captures the system's hybrid character through three internal-scope swimlane activity diagrams (data-driven view) and a state diagram with state-stimulus table (event-driven view), with cross-references back to the context-scope activity diagram in Part I.